

1. หลักการออกแบบ

อธิบายว่าใช้ SOLID ส่วนไหนบ้างและเกี่ยวข้องกับ Entity อย่างไร

S — Single Responsibility Principle แต่ละคลาสมีหน้าที่หลักของตัวเอง กล่าวคือ Entity ไม่ควรรับผิดชอบเรื่อง HTTP หรือการติดต่อกับข้อมูลโดยตรง ดังตาราง

คลาส	หน้าที่
Product	เก็บข้อมูลสินค้าและความสัมพันธ์กับ Entity อื่น
ProductDetail	เก็บรายละเอียดเพิ่มเติมของสินค้า
Review	เก็บข้อมูลรีวิว
ProductRepository	ติดต่อฐานข้อมูลของ Product
ProductDetailRepository	ติดต่อฐานข้อมูลของ ProductDetail
ReviewRepository	ติดต่อฐานข้อมูลของ Review
ProductService	จัดการ Business Logic ของ Product
ProductController	รับ HTTP Request และส่ง Response/View
DiscountStrategy	กำหนดรูปแบบการคำนวณส่วนลด
DiscountContext	ตัวกลางในการเลือก DiscountStrategy ที่เหมาะสมกับประเภทส่วนลดของสินค้า
MemberDiscountStrategy	คำนวณส่วนลดสมาชิก
SeasonalSaleStrategy	คำนวณส่วนลดเทศกาล

O — Open/Closed Principle เปิดให้เพิ่มความสามารถใหม่ได้ โดยไม่ต้องแก้ไขโค้ดเดิมที่ทำงานอยู่

ในโปรเจกต์นี้มีการใช้ Strategy Pattern เพื่อแยกส่วนลดตามประเภทต่างๆ ได้แก่ MEMBER SEASONAL NONE โดยแยกออกมาเป็นคลาสที่ implements จาก DiscountStrategy เพื่อคำนวณส่วนลด

เช่น

```
public class MemberDiscountStrategy implements DiscountStrategy {
    @Override
    public double calculate(double price) {
        return price * 0.9;
    }
}
```

การออกแบบลักษณะนี้ทำให้สามารถเพิ่มประเภทการคำนวณใหม่ได้โดยไม่กระทบโค้ดเดิม

L — Liskov Substitution Principle คลาสลูกสามารถใช้แทนคลาสแม่ได้

ในโปรเจกต์นี้มีการใช้ DiscountStrategy เป็น interface ทำให้เมื่อ DiscountContext เรียกใช้ calculate() สามารถคำนวณตามประเภทนั้นๆ ได้ ดังภาพ

```
public interface DiscountStrategy {
    double calculate(double price);
}
```

```
// Strategy Pattern
public double getDiscountedPrice() {

    if (price == null) {
        return 0.0;
    }

    return DiscountContext
        .getStrategy(discountType)
        .calculate(price);
}
```

I — Interface Segregation Principle คือ interface มีหน้าที่ที่จำเป็นเท่านั้น

ในโปรเจกต์นี้เขียนให้ DiscountStrategy เป็น interface ที่มีหน้าที่เดียวคือ คำนวณราคา ด้วย calculate() ดังภาพ

```
public interface DiscountStrategy {
    double calculate(double price);
}
```

D — Dependency Inversion Principle โมดูลระดับสูงไม่ควรพึ่งพาโมดูลระดับต่ำ

ในโปรเจกต์นี้จึงออกแบบให้ ProductController ไม่มีการสร้าง ProductService และ ProductService ไม่มีการสร้าง ProductRepository กับ ProductDetailRepository แต่ใช้ Constructor Injection เพื่อให้ Spring เป็นผู้สร้างอ็อบเจกต์และส่ง Dependency มาให้ ดังภาพ

```
@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    // Constructor Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String listProducts(Model model) {
        model.addAttribute("products", productService.getAllProducts());
        return "products/list";
    }
}
```

```

@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final ProductDetailRepository productDetailRepository;

    public ProductService(ProductRepository productRepository, ProductDetailRepository productDetailRepository) {
        this.productRepository = productRepository;
        this.productDetailRepository = productDetailRepository;
    }

    // Read All
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
}

```

SOLID เกี่ยวข้องกับ Entity เพราะทำให้หลีกเลี่ยงการนำ business mechanism ที่ซับซ้อนมากมายใส่ใน Entity ส่งผลให้แต่ละคลาสมีหน้าที่ชัดเจนและง่ายต่อการปรับปรุงโค้ดในภายหลัง

อธิบายความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

One-to-One — 1:1 เหมาะกับกรณีที่ข้อมูลเป็น รายละเอียดเฉพาะของ Entity หลักหนึ่งตัว เช่น

Product 1 ————— 1 ProductDetail หมายถึง Product หนึ่งรายการมี ProductDetail หนึ่งรายการ

One-to-Many — 1:N เหมาะกับข้อมูลที่สามารถเกิดขึ้นซ้ำหลายรายการต่อ Entity หลัก เช่น

Product 1 ————— N Review หมายถึง Product หนึ่งรายการสามารถมี Review ได้หลายรายการ

สามารถสรุปความแตกต่างได้ ดังตาราง

	1:1	1:N
ความหมาย	หนึ่งต่อหนึ่ง	หนึ่งต่อหลาย
ตัวอย่าง	Product → ProductDetail	Product → Review
จำนวนลูก	1	หลายรายการ
เหมาะกับ	ข้อมูลรายละเอียดเฉพาะ	ข้อมูลที่เกิดได้หลายรายการ
JPA	@OneToOne	@OneToMany

เขียนโค้ดในคลาส Product ดังนี้

```

// — 1:1 กับ ProductDetail —
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "detail_id", referencedColumnName = "id")
private ProductDetail detail;

// — 1:N กับ Review —
@OneToMany(mappedBy = "product", cascade = CascadeType.ALL)
private List<Review> reviews = new ArrayList<>();

```

อธิบาย Strategy Pattern สำหรับคำนวณส่วนลด

Strategy Pattern คือรูปแบบการออกแบบที่ใช้เมื่อระบบมี วิธีการทำงานหลายแบบสำหรับปัญหาเดียวกัน โดยแยกแต่ละวิธี ออกเป็น class ของตัวเอง แล้วให้โปรแกรมเลือกจะใช้วิธีไหนในขณะทำงาน

ในโปรเจกต์นี้มี 3 ส่วนสำคัญคือ

ส่วน	คลาส	หน้าที่
Strategy	DiscountStrategy	กำหนดมาตรฐานว่าการคำนวณส่วนลดต้องมี calculate()
	MemberDiscountStrategy	คำนวณส่วนลดสมาชิก
	SeasonalSaleStrategy	คำนวณส่วนลดเทศกาล
	NoDiscountStrategy	คืนค่าราคารอกรณไม่มีส่วนลด
Context	DiscountContext	เลือก Strategy ที่เหมาะสม
Entity	Product	เรียกใช้ Strategy เพื่อหาราคาหลังส่วนลด

โดย DiscountStrategy เป็น interface ที่มีเมธอด calculate() เพื่อให้คลาสอื่นนำไป implements ต่อ ดังภาพ

```
public interface DiscountStrategy {
    double calculate(double price);
}
```

NoDiscountStrategy MemberDiscountStrategy และ SeasonalSaleStrategy เป็นคลาสลูกที่ implements เมธอด calculate() เพื่อคืนค่าราคาตามส่วนลดแต่ละประเภท เขียนเป็นโค้ดได้ ดังนี้

```
public class NoDiscountStrategy implements DiscountStrategy {
    @Override
    public double calculate(double price){
        return price;
    }
}
```

```
public class MemberDiscountStrategy implements DiscountStrategy {
    @Override
    public double calculate(double price) {
        return price * 0.9;
    }
}
```

```
public class SeasonalSaleStrategy implements DiscountStrategy {
    @Override
    public double calculate(double price) {
        return price * 0.8;
    }
}
```

DiscountContext ทำหน้าที่เป็น ตัวกลางในการเลือก Strategy ดังภาพ

```
public class DiscountContext {
    public static DiscountStrategy getStrategy(String type) {
        if (type == null) {
            return new NoDiscountStrategy();
        }
        switch (type.toUpperCase()) {
            case "MEMBER":
                return new MemberDiscountStrategy();
            case "SEASONAL":
                return new SeasonalSaleStrategy();
            default:
                return new NoDiscountStrategy();
        }
    }
}
```

ภาพคลาส DiscountContext

การนำ Strategy ไปใช้ใน Product เป็นดังภาพ

```
// Strategy Pattern
public double getDiscountedPrice() {

    if (price == null) {
        return 0.0;
    }

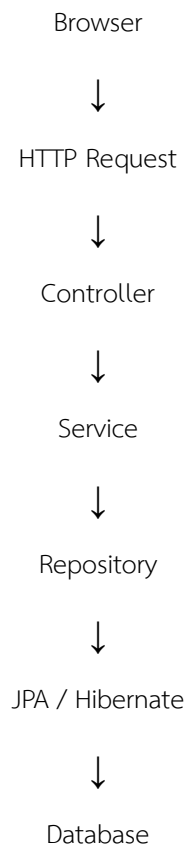
    return DiscountContext
        .getStrategy(discountType)
        .calculate(price);
}
```

ภาพเมธอด getDiscountedPrice ภายในคลาส Product

โดยภายใน Product มี price และ discountType ดังนั้นจึงใช้ DiscountContext เพื่อเลือก Strategy ตาม discountType ที่มี จากนั้นจึงใช้ Strategy คำนวณราคาหลังลดด้วยส่วนลดผ่านเมธอด calculate()

อธิบาย Execution Flow ตั้งแต่ HTTP Request → DB

Execution Flow สามารถเขียนได้ดังนี้



ตัวอย่าง กรณีแก้ไขสินค้า POST /products/update/{id}

1. Browser → HTTP Request

ผู้ใช้งานกรอกข้อมูลสินค้าในหน้าเว็บ แล้วกดปุ่ม “บันทึกการแก้ไข” จากนั้น Browser จะส่ง HTTP Request ไปยัง Server เช่น POST /products/update/1 พร้อมข้อมูลที่ได้จากฟอร์มกรอกข้อมูล

2. HTTP Request → Controller

Spring Boot จะตรวจสอบ URL และ HTTP Method ว่าตรงกับ Controller ตัวไหน เนื่องจากใน ProductController มี @PostMapping("/update/{id}") ตรงกับ Request ดังนั้น Spring จึงเรียกเมธอด updateProduct โดย Spring จะนำข้อมูลจากฟอร์มมาสร้างหรือเติมข้อมูลลงในอ็อบเจกต์ Product

```

@PostMapping("/update/{id}")
public String updateProduct(@PathVariable Long id,
                             @ModelAttribute Product product) {
    productService.updateProduct(id, product);
    return "redirect:/products";
}
  
```

ภาพเมธอด updateProduct ภายในคลาส ProductController

3. Controller → Service

หลังจาก Controller รับข้อมูลแล้ว จะเรียกเมธอด updateProduct ผ่าน Service ดังภาพ กล่าวคือ Service ทำหน้าที่เกี่ยวกับ Business Logic

```
@PostMapping("/update/{id}")
public String updateProduct(@PathVariable Long id,
                             @ModelAttribute Product product) {
    productService.updateProduct(id, product);
    return "redirect:/products";
}
```

ภาพเมธอด updateProduct ภายในคลาส ProductController

```
// Update
public Product updateProduct(Long id, Product product) {
    // Find by id
    Product oldProduct = productRepository.findById(id).orElse(null);

    // Update product fields

    oldProduct.setName(product.getName());
    oldProduct.setCategory(product.getCategory());
    oldProduct.setBrand(product.getBrand());
    oldProduct.setStock(product.getStock());
    oldProduct.setDiscountType(product.getDiscountType());
    oldProduct.setPrice(product.getPrice());

    // Update product detail
    updateProductDetail(oldProduct, product.getDetail());

    return productRepository.save(oldProduct);
}

// Update product detail
private void updateProductDetail(
    Product oldProduct,
    ProductDetail newDetail) {
```

ภาพเมธอด updateProduct ภายในคลาส ProductService

4. Service → Repository

Service ไม่ควรติดต่อฐานข้อมูลโดยตรง แต่จะเรียกผ่าน Repository ดังภาพ

```

55 // Update
56 public Product updateProduct(Long id, Product product) {
57     // Find by id
58     Product oldProduct = productRepository.findById(id).orElse(other: null);
59
60     // Update product fields
61
62     oldProduct.setName(product.getName());
63     oldProduct.setCategory(product.getCategory());
64     oldProduct.setBrand(product.getBrand());
65     oldProduct.setStock(product.getStock());
66     oldProduct.setDiscountType(product.getDiscountType());
67     oldProduct.setPrice(product.getPrice());
68
69     // Update product detail
70     updateProductDetail(oldProduct, product.getDetail());
71
72     return productRepository.save(oldProduct);
73 }
74

```

ภาพเมธอด updateProduct ภายในคลาส ProductService

จากภาพ บรรทัดที่ 58 คือการหา Product ที่ตรงกับ id ซึ่งส่งมาจาก Request ส่วน บรรทัดที่ 72 คือการบันทึกข้อมูลที่มีการแก้ไขลงในฐานข้อมูล

5. Repository → JPA / Hibernate

ProductRepository สืบทอดจาก JpaRepository<Product, Long> ดังนั้นจึงไม่จำเป็นต้องเขียน SQL สำหรับคำสั่ง CRUD พื้นฐานเอง เพราะ JPA/Hibernate จะทำงานเบื้องหลังเพื่อค้นหาข้อมูลจาก Database

```

import com.example.demo.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {

}

```

6. JPA / Hibernate → Database

Hibernate จะส่ง SQL ไปยังฐานข้อมูลหรือ Database

2. Code + คำอธิบาย

Entity ทั้ง 3 ตัว (Product, ProductDetail, Review) พร้อมอธิบาย annotation

Product Entity

Product เป็น Entity หลักสำหรับเก็บข้อมูลสินค้า เช่น ชื่อสินค้า ราคา จำนวนคงเหลือ และประเภทส่วนลด ดังภาพ

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String brand;
    private int stock;
    private Double price;
    private String discountType;

    // — 1:1 กับ ProductDetail —
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    // — 1:N กับ Review —
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL)
    private List<Review> reviews = new ArrayList<>();
```

Annotation

- @Entity เพื่อบอก JPA ว่าคลาสนี้เป็น Entity ที่ต้องนำไป map กับฐานข้อมูล
- @Table(name = "products") กำหนดว่า Entity Product จะเชื่อมกับตารางชื่อ products ถ้าไม่กำหนด @Table JPA อาจใช้ชื่อ class เป็นชื่อตารางตาม naming strategy ที่ตั้งไว้
- @Id private long id; เป็นการกำหนดให้ id เป็น Primary Key ของ Entity
- @GeneratedValue(strategy = GenerationType.IDENTITY) กำหนดให้ Database เป็นผู้สร้างค่า id อัตโนมัติ
- @OneToOne private ProductDetail detail; เป็นการบอกว่า Product มีความสัมพันธ์แบบ หนึ่งต่อหนึ่ง กับ ProductDetail
- @JoinColumn เป็นการกำหนด Column ที่ใช้เชื่อมความสัมพันธ์ ในกรณีนี้ตาราง products จะมี detail_id ซึ่งอ้างอิงไปยัง products_detail.id

- @OneToMany กำหนดให้ Product หนึ่งตัวสามารถมี Review ได้หลายรายการ
 - mappedBy หมายถึงความสัมพันธ์นี้ถูกควบคุมโดย field product ที่อยู่ใน Review เนื่องจาก Review มี private Product product; ดังนั้น Review เป็นฝั่งที่รู้ว่าตัวเองเป็น Review ของ Product ตัวไหน

```
// FK อยู่ฝั่ง Many เสมอ
@ManyToOne
@JoinColumn(name = "product_id")
private Product product;
```

ภาพตัวแปรในคลาส Review

- cascade = CascadeType.ALL หมายถึงการดำเนินการบางอย่างกับ Product สามารถ cascade ไปยัง Entity ที่สัมพันธ์กันได้ เช่น การ save Product ที่มี ProductDetail ใหม่ อาจทำให้ ProductDetail ถูก persist ตามไปด้วย

ProductDetail Entity

ProductDetail เป็น Entity ใช้เก็บรายละเอียดเพิ่มเติมของสินค้า ดังภาพ

```
@Entity
@Table(name="product_details")
public class ProductDetail {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;
    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    // Inverse Side
    @OneToOne(mappedBy="detail")
    private Product product;
```

Annotation

- @Entity เพื่อบอก JPA ว่าคลาสนี้เป็น Entity ที่ต้องนำไป map กับฐานข้อมูล
- @Table(name = " products_details") กำหนดว่า Entity ProductDetail จะเชื่อมกับตารางชื่อ product_details
- @Id private long id; เป็นการกำหนดให้ id เป็น Primary Key ของ Entity
- @GeneratedValue(strategy = GenerationType.IDENTITY) กำหนดให้ Database เป็นผู้สร้างค่า id อัตโนมัติ
- @OneToOne(mappedBy = "detail") เป็นการบอกว่า ProductDetail มีความสัมพันธ์แบบ หนึ่งต่อหนึ่ง กับ Product ส่วนคำคัญคือ mappedBy = "detail" เพราะหมายถึงฝั่ง ProductDetail เป็น inverse side ส่วนฝั่ง Product เป็นฝั่งที่กำหนดความสัมพันธ์

Review Entity

Review ใช้เก็บข้อมูลการรีวิวสินค้า เช่น ผู้รีวิว คะแนน ความคิดเห็น และวันที่รีวิว ดังภาพ

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;
    private Integer rating;
    private String comment;
    private LocalDate reviewDate;

    // FK อยู่ฝั่ง Many เสมอ
    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;
```

Annotation

- @Entity เพื่อบอก JPA ว่าคลาสนี้เป็น Entity ที่ต้องนำไป map กับฐานข้อมูล
- @Table(name = "reviews") กำหนดว่า Entity Review จะเชื่อมกับตารางชื่อ reviews
- @Id private long id; เป็นการกำหนดให้ id เป็น Primary Key ของ Entity
- @GeneratedValue(strategy = GenerationType.IDENTITY) กำหนดให้ Database เป็นผู้สร้างค่า id อัตโนมัติ
- @ManyToOne หมายถึง Review หลายรายการสามารถเป็นของ Product หนึ่งรายการ
- @JoinColumn(name = "product_id") เป็นการกำหนด Foreign Key ของความสัมพันธ์

Service และ Controller พร้อมอธิบาย Constructor Injection

คลาส	หน้าที่
Controller	รับ HTTP Request และส่ง Response/View
Service	จัดการ Business Logic
Repository	ติดต่อ Database

ProductService เป็นชั้นที่อยู่ระหว่าง Controller กับ Repository มีหน้าที่จัดการการทำงานของสินค้า เช่น เพิ่ม แสดง แก้ไข และลบสินค้า มี Constructor Injection คือ ต้องรับ ProductRepository และ ProductDetailRepository ผ่าน Constructor ซึ่งการเขียนแบบนี้ทำให้ ProductService ไม่ต้องสร้าง Repository เอง ดังภาพ

```

@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final ProductDetailRepository productDetailRepository;

    // Constructor Injection
    public ProductService(ProductRepository productRepository, ProductDetailRepository productDetailRepository) {
        this.productRepository = productRepository;
        this.productDetailRepository = productDetailRepository;
    }
}

```

ProductController ทำหน้าที่รับ HTTP Request จาก Browser แล้วเรียก ProductService มี Constructor Injection คือ ต้องรับ ProductService ผ่าน Constructor ซึ่งการเขียนแบบนี้ทำให้ ProductController ไม่ต้องสร้าง Service เอง ดังภาพ

```

@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    // Constructor Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }
}

```

3. ภาพหน้าจอ

CREATE - หน้าเพิ่มสินค้าขึ้นใหม่

Database ก่อนกดบันทึกสินค้า

ตาราง products

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Sony	Headphones	MEMBER	Sony WH-1000XM5 (673380299-1 SEC...	9990	10	1

ตาราง product_details

	id [PK] bigint	description character varying (255)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1	หูฟังไร้สายแบบครอบหู พร้อมระบบตัดเสียงรบกวนอัจฉ...	20.0 x 18.5 x 7.5 cm	Malaysia	1 ปี	0.21

ตาราง reviews

	id [PK] bigint	comment character varying (255)	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	1	เสียงดีมาก ระบบตัดเสียงรบกวนทำงานได้ยอดเยี่ยม...	5	[null]	ใจดี สุดสวย	1

READ - รายการสินค้าในตาราง

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	ส่วนลด (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	Sony WH-1000XM5 (673380299-1 SEC 2)	Headphones	Sony	10	9990.00	MEMBER	8991.00	1 ปี	0.21	1 รีวิว	แก้ไขลบ
2	Keychron K8 Pro (673380299-1 SEC 2)	Keyboard	Keychron	15	3290.00	SEASONAL	2632.00	1 ปี	0.97	1 รีวิว	แก้ไขลบ

Database หลักกฉบับที่สินค้า

ตาราง products

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Sony	Headphones	MEMBER	Sony WH-1000XM5 (673380299-1 S...	9990	10	1
2	2	Keychron	Keyboard	SEASONAL	Keychron K8 Pro (673380299-1 SEC...	3290	14	2

ตาราง product_details

	id [PK] bigint	description character varying (255)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1	หูฟังไร้สายแบบครอบหู พร้อมระบบตัดเสียงรบกวนอัจฉ...	20.0 x 18.5 x 7.5 cm	Malaysia	1 ปี	0.21
2	2	คีย์บอร์ด Mechanical แบบไร้สาย รองรับ Bluetooth	35.9 x 12.7 x 3.8 cm	China	1 ปี	0.97

ตาราง reviews

	id [PK] bigint	comment character varying (255)	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	1	เสียงดีมาก ระบบตัดเสียงรบกวนทำงานได้ยอดเยี่ยม	5	[null]	ใจดี สุดสวย	1
2	2	พิมพ์สนุก ปุ่มกดดี และเชื่อมต่อ Bluetooth ได้สะดวก	4	[null]	นี่	2

UPDATE - หน้าฟอร์มแก้ไข

← → ↺ 📄 🌐 localhost:8080/products/edit/2

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

Keychron K8 Pro (673380299-1 SEC 2)

หมวดหมู่

Keyboard

ยี่ห้อ

Keychron

จำนวนในคลัง

14

ราคาปกติ (บาท)

3290.0

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

คีย์บอร์ด Mechanical แบบไร้สาย รองรับ Bluetooth

ระยะเวลาประกัน

1 ปี

น้ำหนัก (กิโลกรัม)

0.97

ขนาด

35.9 x 12.7 x 3.8 cm

ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก

DELETE - ยืนยันลบ

← → ↺ 📄 🌐 localhost:8080/products/delete/2

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้หรือไม่? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า

Keychron K8 Pro (673380299-1 SEC 2)

หมวดหมู่

Keyboard

ยี่ห้อ

Keychron

ราคา

3290.00 บาท

รายละเอียด

คีย์บอร์ด Mechanical แบบไร้สาย รองรับ Bluetooth

รับประกัน

1 ปี

ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

DELETE – รายการสินค้าหลังลบ

Product Shop Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	Sony WH-1000XM5 (673380299-1 SEC 2)	Headphones	Sony	10	9990.00	MEMBER	8991.00	1 ปี	0.21	1 รีวิว	แก้ไข ลบ

Database หลังลบสินค้า

ตาราง products

	id [PK] bigint	description character varying (255)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1	หูฟังไร้สายแบบครอบหู พร้อมระบบตัดเสียงรบกวนอัจฉ...	20.0 x 18.5 x 7.5 cm	Malaysia	1 ปี	0.21

ตาราง product_details

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Sony	Headphones	MEMBER	Sony WH-1000XM5 (673380299-1 SEC...	9990	10	1

ตาราง reviews

	id [PK] bigint	comment character varying (255)	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	1	เสียงดีมาก ระบบตัดเสียงรบกวนทำงานได้ยอดเยี่ยม...	5	[null]	ใจดี สุดสวย	1